Name : Vansh Sharma

Roll No : 2401109

Batch : T22

Subject : ASD&D

Experiment-04

Aim:

To understand Continuous Integration and Jenkins installation.

This experiment maps to **LO3**.

LO3:

Install, configure, and use Jenkins to automate software development tasks.

Theory:

Continuous Integration (CI) is a software development practice in which developers frequently integrate their code into a shared repository. Every code change is automatically built and tested, allowing early detection of errors and improving software quality.

Jenkins is an open-source automation server developed in Java that supports Continuous Integration and Continuous Delivery (CI/CD). It automates different stages of the software development lifecycle such as building, testing, deployment, and monitoring. Jenkins supports hundreds of plugins, allowing integration with tools like Git, GitHub, Maven, Gradle, Docker, Kubernetes, and cloud platforms.

Jenkins provides a web-based interface that allows users to configure projects, monitor build status, generate reports, and manage automation workflows.

In this experiment, Jenkins is installed locally using the Jenkins WAR file. The installation process includes configuring Java, launching the Jenkins server, unlocking Jenkins using the initial administrator password, installing recommended plugins, creating an administrator account, and accessing the Jenkins dashboard.

The Jenkins installation process consists of the following steps:

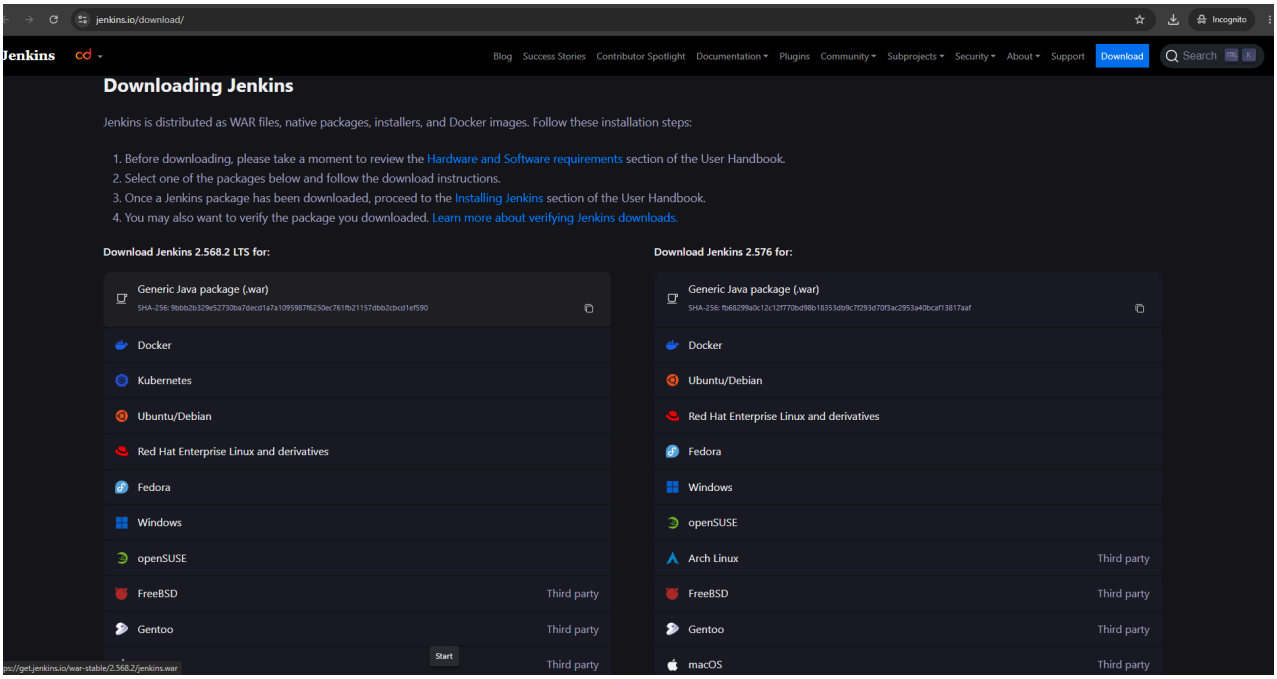
Java Development Kit (JDK)

- Required to run Jenkins because Jenkins is Java-based.
- Verify Java installation using the command:

```
PS C:\Users\15L\Downloads> java -version
java version "21.0.2" 2024-01-16 LTS
Java(TM) SE Runtime Environment (build 21.0.2+13-LTS-58)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.2+13-LTS-58, mixed mode, sharing)
```

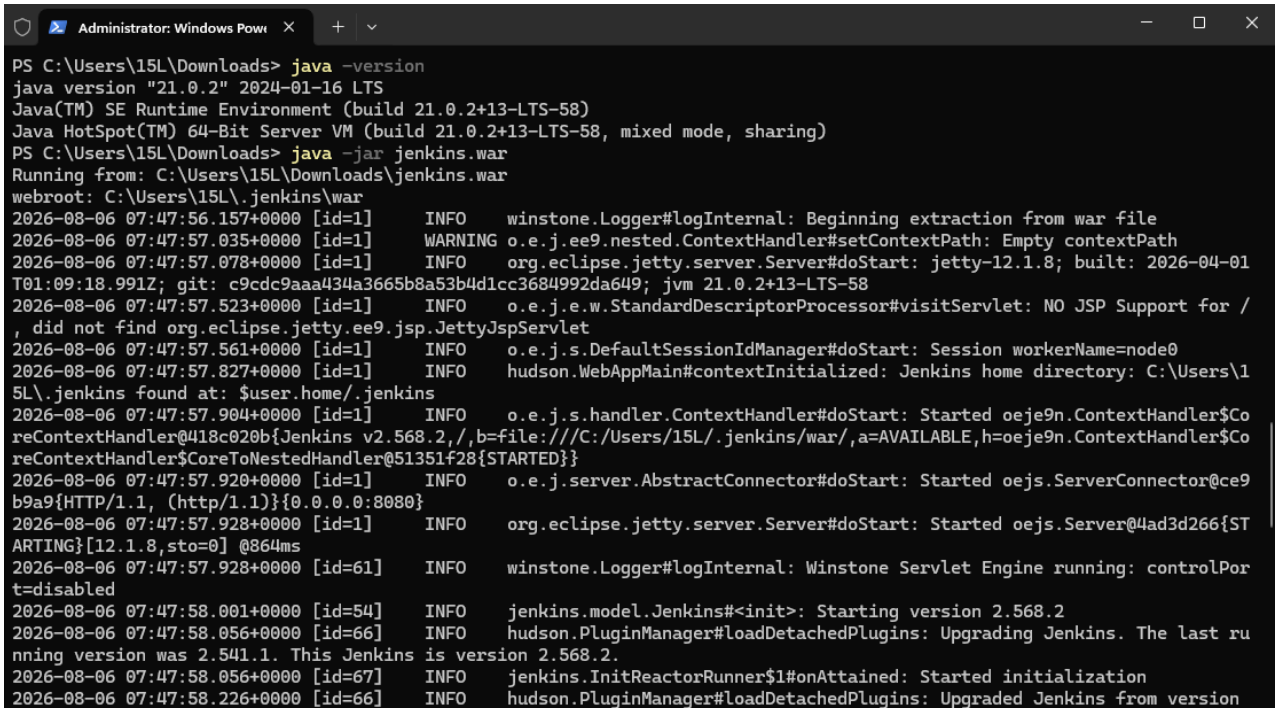
Download Jenkins

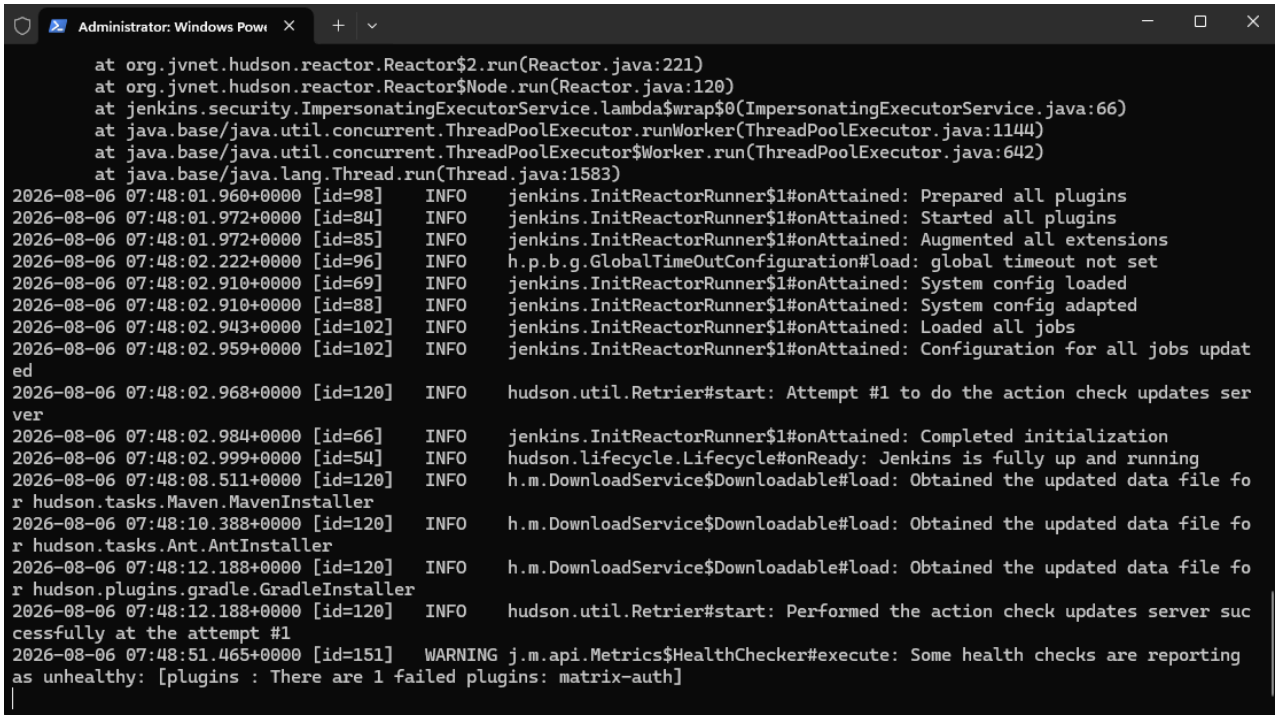
- Download the latest Jenkins WAR file from the official Jenkins website.
- Save the file in the desired directory.



Launch Jenkins

- Open Command Prompt or PowerShell.
- Navigate to the directory containing the Jenkins WAR file.
- Run Jenkins using:





Access Jenkins

- Open a web browser.
- Visit:

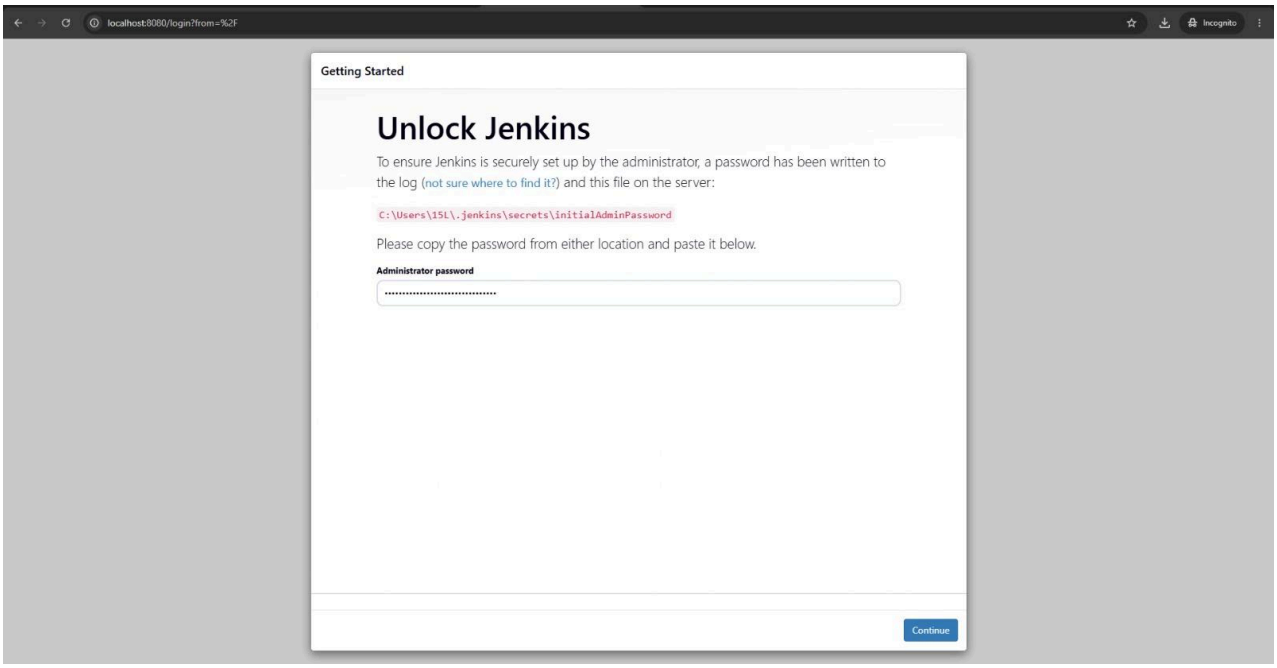
http://localhost:8080

- Jenkins Setup Wizard appears.

Unlock Jenkins

- Retrieve the initial administrator password from:

C:\Users\<Username>\.jenkins\secrets\initialAdminPassword

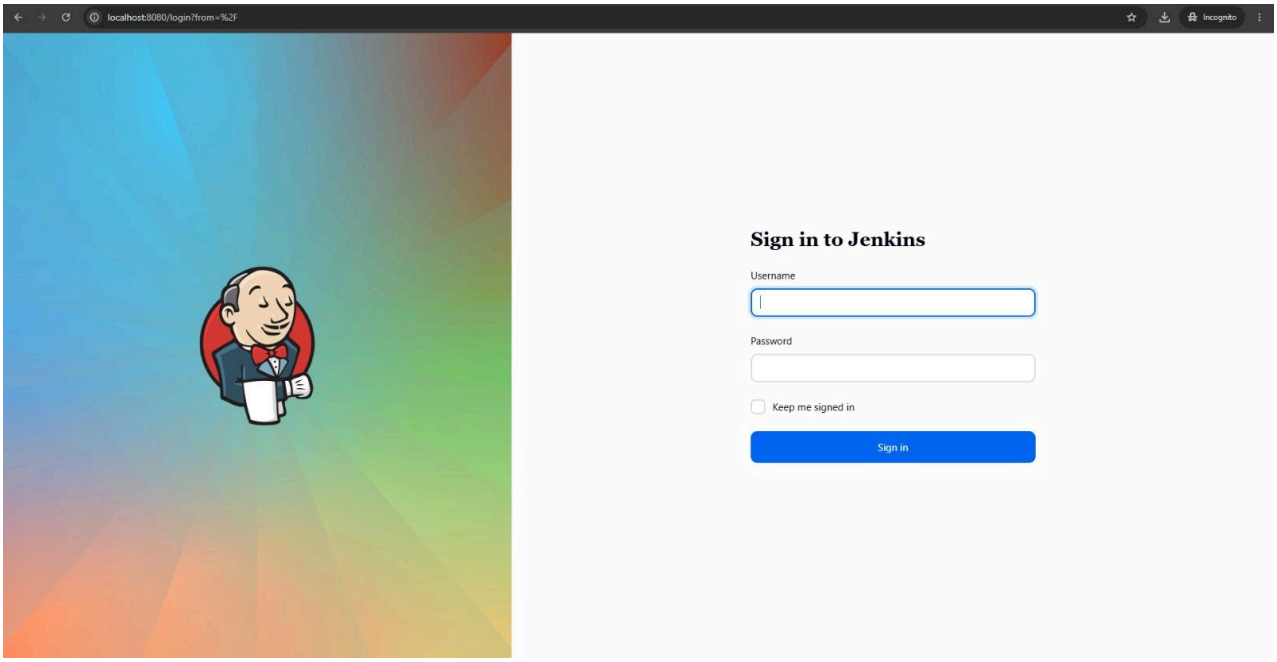
- Paste the password into the Jenkins setup page.
- 

Install Plugins

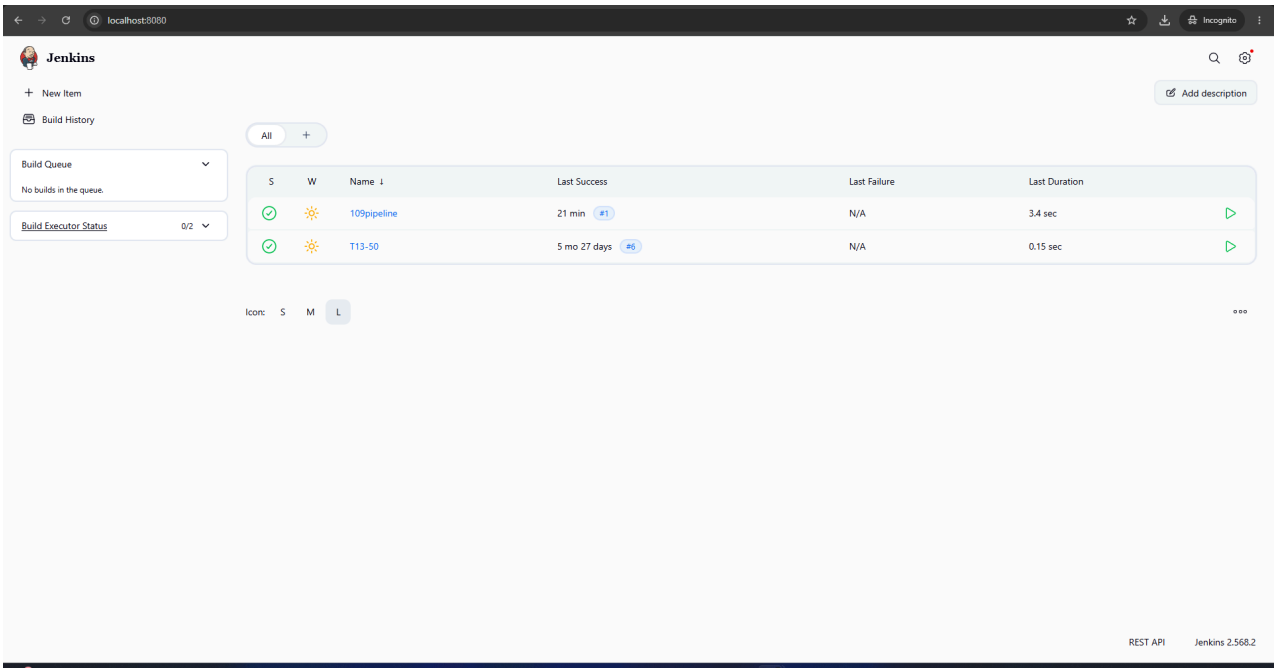
- Select **Install Suggested Plugins** to install the default Jenkins plugins.

Create Admin User

- Enter username, password, full name, and email address.
- Save the credentials.

- 

Jenkins Dashboard

- Login using the created administrator account.
- Jenkins Dashboard is displayed.
- 

A Jenkins Pipeline is a sequence of automated stages that define the software development workflow. It enables developers to automate the processes of building, testing, and deploying applications. Pipelines are written as code, making them easier to maintain, modify, and reuse.

Jenkins supports **Pipeline as Code**, where the complete workflow is stored in a text file called a **Jenkinsfile**. This allows the pipeline configuration to be version-controlled along with the application source code.

Jenkins provides two types of pipelines:

Declarative Pipeline

- Simple and structured syntax.
- Easy to read and maintain.
- Recommended for beginners.
- Uses the `pipeline {}` block.

Scripted Pipeline

- More flexible and powerful.
- Uses Groovy scripting.
- Suitable for complex workflows.
- Uses the `node {}` block.

In this experiment, a simple Declarative Pipeline is created to display a message in the Jenkins Console Output. The pipeline consists of an **Agent**, **Stages**, and **Steps**. The **Agent** specifies where the pipeline runs, **Stages** divide the workflow into logical phases, and **Steps** define the commands executed during each stage.

Advantages of Jenkins Pipeline

- Automates CI/CD processes.
- Supports Pipeline as Code.
- Easy to maintain and reuse.
- Improves software quality.
- Reduces manual effort.
- Supports version control.
- Simplifies debugging.
- Enables continuous software delivery.

Pipeline Script

```
pipeline {
   agent any

   stages {
      stage('Hello') {
         steps {
            echo 'Hello, Jenkins!'
         }
      }
   }
}
```

Implementation Steps

Step 1:

Open the **Jenkins Dashboard** in your web browser.

Step 2:

Click **New Item**.

Step 3:

Enter the project name.

Example:

109pipeline

Step 4:

Select **Pipeline** and click **OK**.

Step 5:

Scroll to the **Pipeline** section.

Step 6:

Select **Pipeline Script** under the **Definition** option.

Step 7:

Paste the following Declarative Pipeline script:

```
pipeline {
   agent any

   stages {
      stage('Hello') {
         steps {
            echo 'Hello, Jenkins!'
         }
      }
   }
}
```

Step 8:

Click **Save**.

Step 9:

Click **Build Now**.

Step 10:

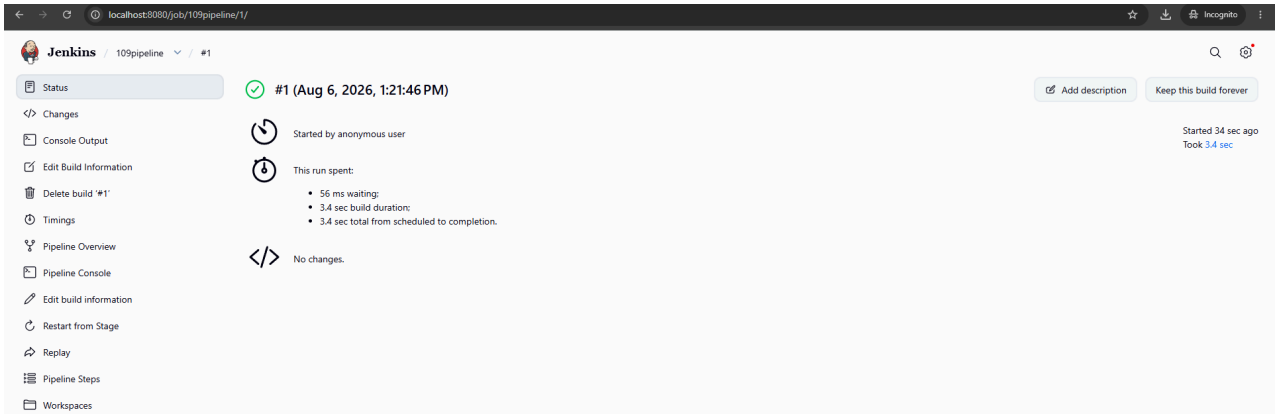
Select the latest build.

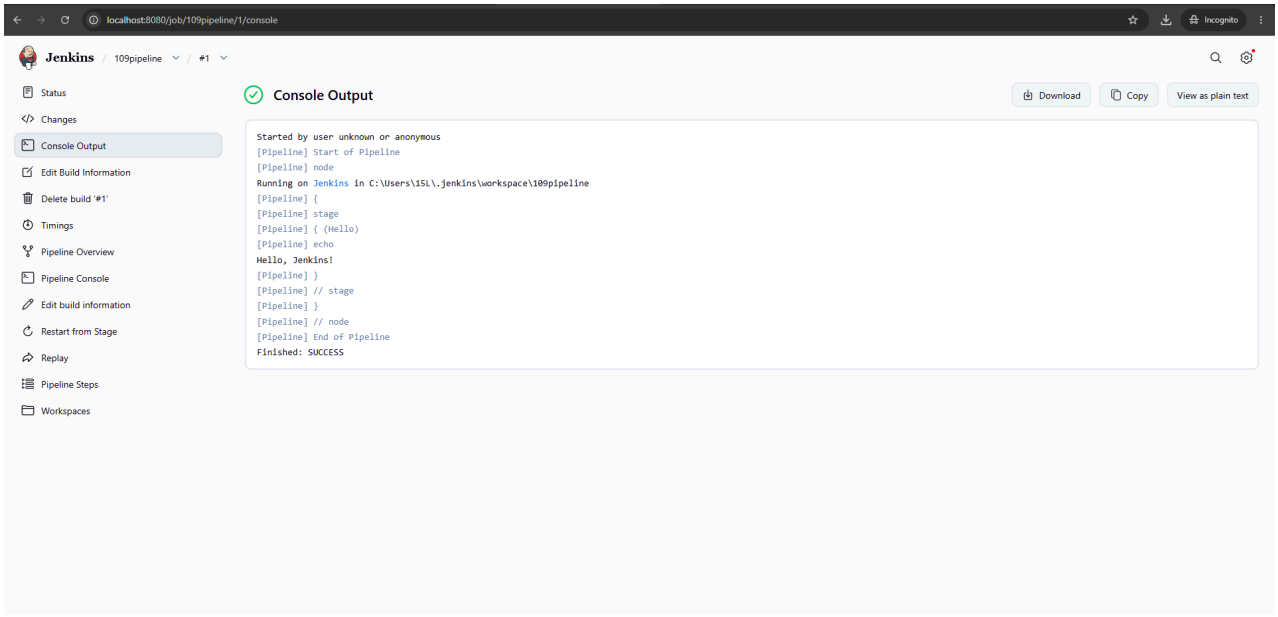
Step 11:

Click **Console Output**.

Step 12:

Verify that the pipeline executes successfully and displays:





Commands

Command / Action	Purpose
Open Jenkins Dashboard	Opens the Jenkins web interface.
New Item	Creates a new Jenkins project.
Pipeline	Selects the Pipeline project type.
Pipeline Script	Opens the editor for writing pipeline code.
Save	Saves the pipeline configuration.

Build Now	Executes the pipeline.
Console Output	Displays the execution log of the pipeline.
Replay *(optional)*	Executes the pipeline again after editing the script.

Output

- Pipeline project created successfully.
- Pipeline executed successfully.
- Console Output displayed:

Started by user admin

[Pipeline] Start of Pipeline

[Pipeline] stage

[Pipeline] { (Hello)

Hello, Jenkins!

[Pipeline] }

Finished: SUCCESS

Conclusion

From this experiment, I learned how to create and execute a Jenkins Pipeline using Declarative Pipeline syntax. I understood the concept of Pipeline as Code and learned how Jenkins automates software development workflows through different stages. I also learned how to create a pipeline project, execute it using **Build Now**, and verify the execution through the **Console Output**. This experiment demonstrated how Jenkins Pipelines simplify Continuous Integration (CI) and Continuous Delivery (CD) by automating repetitive development tasks.